

Documentação: Trabalho Prático 2

Modelagem de Redes Sociais Acadêmicas

Amanda Vasconcelos Pereira
Matrícula: 2025049387

1 Introdução

Nesse trabalho foi desenvolvida uma plataforma chamada ConectaDCC, que é uma rede social acadêmica. O programa funciona por meio de dois grafos que são responsáveis por armazenar as relações de interesse entre usuários e temas e as conexões entre os usuários.

Grafo de Temas: grafo bipartido, não direcionado e não ponderado que mapeia a relação de interesse de usuários por temas.

Grafo Social: grafo direcionado e não ponderado que modela quem cada usuário segue e quem o segue de volta (caracterizando relações de amizade mútuas).

O sistema permite alterar a forma de armazenamento dos grafos ao longo da execução (listas de adjacência ou matriz de adjacência), listar os temas, os seguidores e os seguidos de um usuário, consultar a relação de seguimento entre dois usuários e se um tema é de interesse de um usuário e listar a popularidade de um tema (quantidade de usuário com interesse nele).

Dessa forma, este relatório documenta as decisões de projeto adotadas na elaboração dos algoritmos na linguagem C++11, detalhando as estruturas básicas utilizadas para as operações de cadastro, remoção e consultas (TADs), além de apresentar uma análise experimental comparativa de desempenho e uso de memória sob diferentes cargas de uso da plataforma.

2 Método

A especificação do projeto foi solucionada através da criação de Tipos Abstratos de Dados (TADs) customizados em C++11, utilizando ponteiros brutos, referências e alocação dinâmica gerenciada manualmente. O programa é composto pelas classes Tema, Usuario, Dicionario e Grafo, integradas de forma sequencial no arquivo principal main.cpp.

2.1 Modelagem dos Tipos Abstratos de Dados (TADs)

A modularização do sistema baseia-se em quatro TADs fundamentais que cooperam entre si e uma estrutura:

- **TAD Tema:** Armazena os atributos de um tópico de interesse. Contém um identificador inteiro, uma string para o nome e um caractere para o tipo.

- **TAD Usuário:** Armazena as informações básicas de um membro da plataforma. Contém o identificador único, o nome do usuário e sua idade. Além disso, gerencia as preferências de tema por meio de um ponteiro `idsTemas` para um arranjo de inteiros alocado dinamicamente no construtor, cujo tamanho exato é delimitado por `numTemas`.
- **Estrutura de Nós:** Para viabilizar que uma única classe de grafo armazene vértices de naturezas distintas, foi empregado uma struct cujas instâncias carregam o identificador fornecido pela entrada, a categoria do objeto e um índice sequencial físico interno. Adicionalmente, possui um ponteiro `proximo` para encadeamento, atuando diretamente na representação por listas.
- **TAD Dicionário:** Atua como o núcleo indexador e gerenciador de memória do sistema. Aproveitando a garantia de que as entradas fornecem identificadores sequenciais contíguos a partir de zero, a classe mapeia os registros em tempo constante $O(1)$ utilizando cinco arranjos de ponteiros paralelos.
- **TAD Grafo:** É a estrutura unificada responsável por gerenciar toda a topologia de rede. Ela encapsula os atributos privados `direcionado`, `modoArmazenamento`, `numVertices` e `capVertices` (para controle de redimensionamento). Como principal característica, possui os ponteiros brutos `listaAdjacencia` e `matrizAdjacencia`, além de um arranjo `mapeamentoVertices`. Essa modelagem permite que a classe execute tanto operações de caminhos e adjacências quanto a transição completa de sua estrutura interna sob demanda.

2.2 Redimensionamento Dinâmico do Dicionário

Como os limites máximos de usuários e temas não são conhecidos a priori, o dicionário inicia com uma capacidade padrão de 10 elementos. Quando um método de inserção intercepta um identificador maior ou igual à capacidade vigente, as rotinas `redimensionarUsuarios` ou `redimensionarTemas` são disparadas:

1. Um novo bloco de ponteiros com o dobro do tamanho necessário é alocado.
2. A inicialização utiliza a sintaxe de parênteses, forçando o compilador a preencher todas as novas posições com `nullptr` automaticamente.
3. Os ponteiros antigos são copiados para a nova área.
4. Os arranjos defasados são liberados com `delete[]` e as referências internas da classe são atualizadas para a nova área expandida.

2.3 Estrutura Intercambiável do TAD Grafo

O TAD Grafo foi projetado para operar possibilitando a alteração do seu modo de armazenamento através do atributo `modoArmazenamento` (`'L'` ou `'M'`). Vértices são inseridos dinamicamente por meio de `adicionarVertice`, que gera os nós vinculando o `idLogico` a um `idInterno` físico e sequencial de base zero.

2.3.1 Representação por Lista de Adjacência

É composta por um arranjo de ponteiros, indexado pelo `idInterno` do vértice de origem, onde cada posição aponta para o início de uma lista encadeada de nós vizinhos.

Inserção de Arestas: A função `adicionarAresta` instancia uma cópia do nó de destino e realiza a inserção diretamente no topo da lista encadeada correspondente. No Grafo de Temas, o processo é imediatamente espelhado inserindo a volta.

Remoção de Arestas: No comando `removerAresta`, o programa executa uma varredura linear clássica na lista encadeada rastreando o vértice alvo. O nó correspondente é retirado da corrente ajustando os ponteiros do elemento anterior e deletado da memória.

2.3.2 Representação por Matriz de Adjacência

Aloca dinamicamente uma matriz quadrada de inteiros de tamanho `capVertices` $\times$ `capVertices`. A presença de arestas é indicada binariamente atribuindo o valor 1 na coordenada correspondente, enquanto a ausência ou remoção é definida por 0. A verificação de adjacência nesta estrutura é direta e possui um tempo estritamente constante.

2.4 Mecanismo Alternância de Modo

A preservação integral do estado lógico dos grafos durante a transição estrutural é garantida pelo método `alternarModo`.

Migração para Matriz ('L' $\rightarrow$ 'M'): O grafo aloca a estrutura matricial limpa preenchida com zeros. Na sequência, percorre o vetor de listas encadeadas, para cada nó adjacente visitado, marca a célula respectiva na matriz como 1. Ao término, a função privada `limparLista()` percorre e deleta cada nó das listas encadeadas antigas, bem como o vetor de ponteiros principal.

Migração para Lista ('M' $\rightarrow$ 'L'): O grafo aloca o arranjo de cabeças de lista zerado. Posteriormente, realiza uma varredura bidimensional aninhada sobre a matriz ativa. Sempre que encontra uma célula com valor 1, recupera o nó correspondente a partir do array `mapeamentoVertices` e encadeia uma nova cópia deste no topo da lista do vértice corrente. A estrutura matricial é totalmente desalocada através da função `limparMatriz()`.

2.5 Fluxo do Programa Principal (main)

O fluxo de execução implementa o algoritmo sequencial sugerido. As linhas são lidas e fragmentadas e a busca de entidades é totalmente delegada ao TAD `Dicionario` via ponteiros obtidos pelos IDs numéricos.

Para as operações de consulta que exigem ordenação crescente de identificador, o código adota uma estratégia de ordenação implícita: a varredura é efetuada diretamente sobre os índices contíguos de 0 até `numVertices` do grafo. Como os IDs físicos e lógicos crescem em paralelo, a própria ordem natural do loop garante que a saída seja impressa perfeitamente ordenada, minimizando o custo computacional e simplificando o código.

3 Análise de Complexidade

Nesta seção, é apresentada a análise assintótica de tempo e espaço das estruturas de dados e dos principais procedimentos implementados, considerando os dois modos de

armazenamento exigidos: Lista de Adjacência (L) e Matriz de Adjacência (M).

A análise considera N - número total de usuários cadastrados, M - número total de temas cadastrados e K - quantidade máxima de temas de interesse de um único usuário.

3.1 Complexidade de Espaço (Memória)

3.1.1 Modo Lista de Adjacência (L)

Grafo Social: Possui N vértices. Cada usuário pode seguir até $N - 1$ outros usuários. Se o grafo for denso, o número de arestas é $O(N^2)$, mas se for esparso, é $O(N)$. O armazenamento das listas consome $O(N + A_{social})$ de memória, onde A_{social} é o número de arestas de seguimento.

Grafo de Temas: É um grafo bipartido contendo $(N + M)$ vértices. Como cada usuário possui no máximo K temas, o número total de arestas é limitável por $N \cdot K$. Assim, o espaço ocupado por este grafo é $O(N + M + N \cdot K) = O(M + N \cdot K)$.

Dicionário e TADs: Os arrays dinâmicos de ponteiros indexados diretamente crescem linearmente conforme os IDs aumentam, ocupando $O(N + M)$.

Total (Modo L): $O(N^2 + M)$ no pior caso (grafo social completo), ou $O(N \cdot K + M)$ caso o grafo social seja esparso.

3.1.2 Modo Matriz de Adjacência (M)

Grafo Social: Aloca uma matriz quadrada de dimensões $N \times N$, resultando em um consumo fixo de $O(N^2)$.

Grafo de Temas: Aloca uma matriz quadrada para todos os vértices do grafo bipartido, ou seja, $(N + M) \times (N + M)$. Isso resulta em um consumo fixo de $O((N + M)^2) = O(N^2 + N \cdot M + M^2)$.

Total (Modo M): $O(N^2 + M^2)$, o que torna o uso de memória significativamente maior em cenários onde o número de temas ou usuários cresce muito, independentemente do número real de arestas existentes.

3.2 Complexidade de Tempo

A Tabela abaixo apresenta a comparação da complexidade de tempo de pior caso para cada uma das operações comandadas pela função principal (`main.cpp`), contrastando o modo **Lista de Adjacência (L)** e o modo **Matriz de Adjacência (M)**.

Comando	Modo Lista (L)	Modo Matriz (M)
A	$O(N^2 + M^2)$	$O(N^2 + M^2)$
T	$O(1)$ amortizado	$O(M)$ amortizado
U	$O(K)$ amortizado	$O(N + K)$ amortizado
S / R	$O(N)$	$O(1)$
LT	$O(K)$	$O(K)$
LC / LS / LA	$O(N^2)$	$O(N^2)$
Q	$O(N)$	$O(1)$
G	$O(K)$	$O(1)$
F	$O(N \cdot K)$	$O(N + M)$

3.3 Detalhamento e Justificativa das Operações Principais

3.3.1 Alternar Modo de Armazenamento (A)

Ao migrar de Lista para Matriz, o algoritmo precisa instanciar uma matriz do zero e percorrer a lista de adjacência de cada um dos vértices para preencher as posições correspondentes com 1. No pior caso, varre-se todas as arestas possíveis. Ao migrar de Matriz para Lista, o algoritmo realiza uma varredura completa por todas as células das matrizes ($N \times N$ para o social e $(N + M) \times (N + M)$ para o de temas) para reconstruir as listas encadeadas. Portanto, o custo é dominado pelo tamanho máximo das matrizes: $O(N^2 + M^2)$.

3.3.2 Operações de Escrita Básica (T, U, S, R)

Cadastros (T e U): Inserir um tema ou usuário nos arrays do dicionário custa $O(1)$. Caso o tamanho estoure a capacidade atual, ocorre o redimensionamento (duplicação do tamanho do array), que leva tempo linear em relação à capacidade anterior, gerando um custo $O(1)$ amortizado. Adicionar as K arestas de interesse no modo Lista leva $O(K)$, pois a inserção na lista encadeada é feita sempre no início ($O(1)$ por aresta). No modo Matriz, o redimensionamento exige recriar a matriz quadrada inteira adicionando a nova linha/coluna, o que toma $O(M)$ ou $O(N)$.

Seguir e Remover (S e R): No modo Lista, para verificar a existência ou remover, é necessário percorrer linearmente a lista de adjacência do usuário conectado. No pior caso (um usuário que segue todo mundo), a lista tem tamanho $O(N)$, tornando a operação $O(N)$. No modo Matriz, a inserção, consulta e remoção são feitas via indexação direta de índices, o que garante tempo constante $O(1)$.

3.3.3 Consultas de Listagem (LC, LS, LA)

Para esses comandos, o código varre todos os N usuários cadastrados no sistema através do mapeamento de vértices (`mapeamentoVertices`). Para cada usuário da rede, ele executa a função `existeAresta` para checar se ele se conecta ao usuário consultado.

No modo Lista, `existeAresta` percorre a lista de vizinhos do usuário atual (que pode ter tamanho até $O(N)$). Multiplicando a varredura externa de usuários (N) pelo custo interno da busca na lista ($O(N)$), chegamos a $O(N^2)$ no pior caso.

No modo Matriz, embora o método `existeAresta` seja $O(1)$, o loop principal na função do `main.cpp` ainda itera por todos os N vértices cadastrados. O custo total de tempo computado no algoritmo para gerar a saída ordenada por ID é de $O(N^2)$.

3.3.4 Popularidade do Tema (F)

O comando percorre todos os vértices do Grafo de Temas procurando por nós do tipo 'U' (Usuários). Para cada usuário encontrado, verifica se há uma aresta ligando-o ao tema em questão através de `existeAresta`.

No modo Lista, a checagem percorre a lista de interesses do usuário (tamanho máximo K). Como existem N usuários, o tempo total gasto é de $O(N \cdot K)$.

No modo Matriz, o loop varre todos os vértices do grafo de temas (que totalizam $N + M$). Mesmo que o acesso à matriz seja rápido, o tamanho da varredura sobre o mapeamento de vértices impõe um limite de pior caso de $O(N + M)$ para a checagem.

4 Estratégias de Robustez

4.1 Inicialização Segura e Prevenção de Ponteiros Soltos

Inicialização com `nullptr` via Construtor: No construtor da classe `Dicionario`, a alocação dos arrays de ponteiros utiliza parênteses ao final da expressão `new`. Essa sintaxe garante a inicialização por valor, definindo automaticamente todos os ponteiros internos como `nullptr`.

Aterramento de Ponteiros no Destrutor: Após a aplicação do operador `delete` ou `delete[]` nos destrutores das classes `Usuario`, `Dicionario` e `Grafo`, os ponteiros correspondentes são imediatamente redefinidos para `nullptr`. Isso evita que chamadas acidentais subsequentes tentem acessar endereços inválidos de memória.

4.2 Redimensionamento Dinâmico Controlado

Como os identificadores de usuários e temas são gerados sequencialmente a partir de 0 e armazenados em arrays estáticos de ponteiros de tamanho inicial igual a 10, o sistema realiza um monitoramento ativo do limite dessas estruturas antes de qualquer inserção.

Validação de Limites: Nos métodos `registrarUsuario` e `registrarTema` da classe `Dicionario`, e `adicionarVertice` da classe `Grafo`, o sistema verifica se o identificador recebido ou a quantidade atual de elementos ultrapassa a capacidade alocada (`capUsuarios`, `capTemas` ou `capVertices`).

Estratégia de Expansão Geométrica: Caso o limite seja atingido, os métodos de redimensionamento realocam novos blocos de memória com o dobro da capacidade necessária. Os dados antigos são copiados, os ponteiros antigos são liberados via `delete[]` e os novos arrays assumem o controle, evitando erros de estouro de buffer.

4.3 Proteção contra Falhas de Segmentação nas Consultas

Validação de Índices: Antes de acessar qualquer posição dos arrays internos, os métodos `recuperarUsuario`, `recuperarTema` e os recuperadores de nós validam se o ID fornecido está no intervalo válido $0 \leq id < \text{capacidade}$. Caso o ID seja negativo ou maior/igual à capacidade corrente, o método retorna imediatamente `nullptr`.

Tratamento de Retorno Nulo: Na função `main` e nas operações internas do `Grafo`, existem verificações explícitas (ex: `if (u == nullptr || v == nullptr) return;`). Esse fluxo condicional garante que o programa ignore comandos inconsistentes ou linhas mal-formadas sem interromper sua execução de forma abrupta.

4.4 Prevenção de Vazamento de Memória

O arquivo `main.cpp` responsabiliza-se apenas por destruir as três estruturas principais no fim da execução: `grafoSocial`, `grafoTemas` e `dic`.

A classe `Dicionario`, por possuir a posse real dos objetos, varre seus arrays internos e deleta individualmente cada instância de `Usuario` e `Tema` antes de deletar os arrays de ponteiros em si.

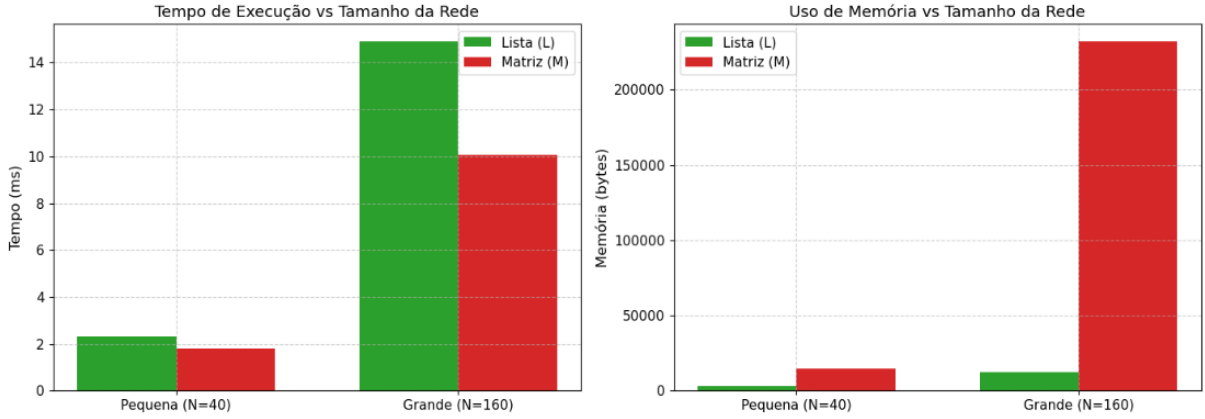
A classe `Grafo` gerencia a limpeza específica de suas listas de adjacência ou matrizes, garantindo que, mesmo quando ocorre a alternância de modo (comando `A`), a estrutura anterior seja completamente limpa da memória, impossibilitando o acúmulo de lixo computacional.

5 Análise Experimental

A análise experimental tem como objetivo avaliar o desempenho prático e a eficiência de espaço das duas representações de grafos implementadas: Lista de Adjacência (L) e Matriz de Adjacência (M). Os testes foram projetados para isolar e medir o impacto de sete parâmetros fundamentais do sistema ConectaDCC: o número total de usuários (n), o número total de temas (m), o número médio de temas por usuário, o número total de relações de seguimento, a densidade do grafo social, e as frequências relativas de atualizações e consultas.

5.1 Teste 1: Impacto do Tamanho da Rede (n, m)

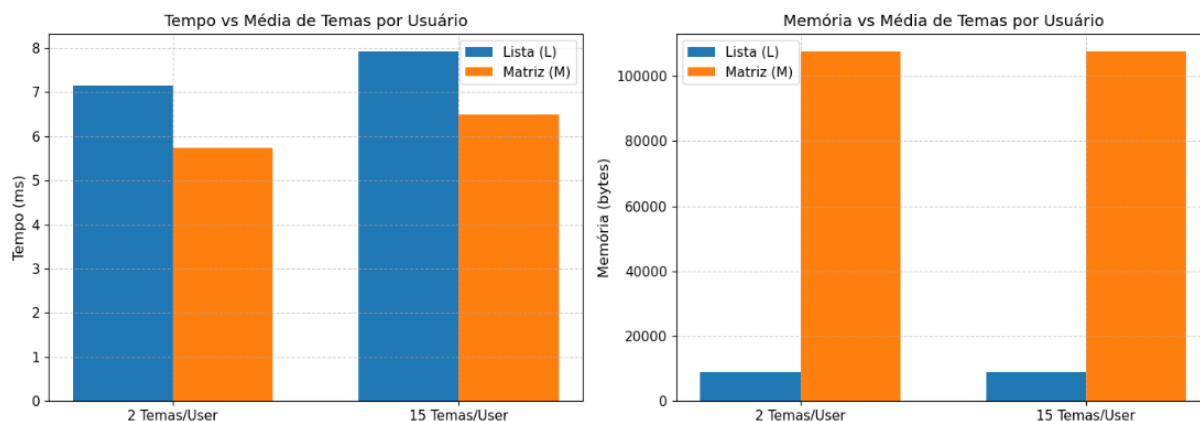
Neste primeiro cenário, avaliou-se o comportamento assintótico das estruturas à medida que a rede social escala em termos de usuários (n) e temas (m). Os testes foram divididos entre uma rede pequena ($n = 40, m = 5$) e uma rede grande ($n = 160, m = 20$), mantendo os demais fatores de carga constantes.



- **Eficiência de Espaço (memória):** Conforme observado no gráfico de barras, o uso de memória no modo Matriz (M) sofreu uma expansão drástica, saltando de 14.500 bytes para 232.000 bytes ao transicionar para a rede grande. Esse comportamento ocorre pois as matrizes exigem alocação de ordem quadrática, ou seja, $\mathcal{O}(V^2)$. Em contrapartida, o modo Lista (L) demonstrou excelente economia de recursos, subindo de 3.080 bytes para apenas 12.320 bytes, uma vez que seu consumo é dinâmico e linearmente dependente do somatório de vértices e arestas reais, mapeado por $\mathcal{O}(V + E)$.
- **Eficiência de Tempo:** O tempo total gasto na simulação completa da Matriz foi discretamente inferior ao da Lista em ambos os tamanhos. Isso ocorre porque o custo de inicialização das matrizes $N \times N$ na memória ainda não superou o ganho obtido pelos acessos imediatos indexados por ponteiros.

5.2 Teste 2: Variação do Número Médio de Temas por Usuário

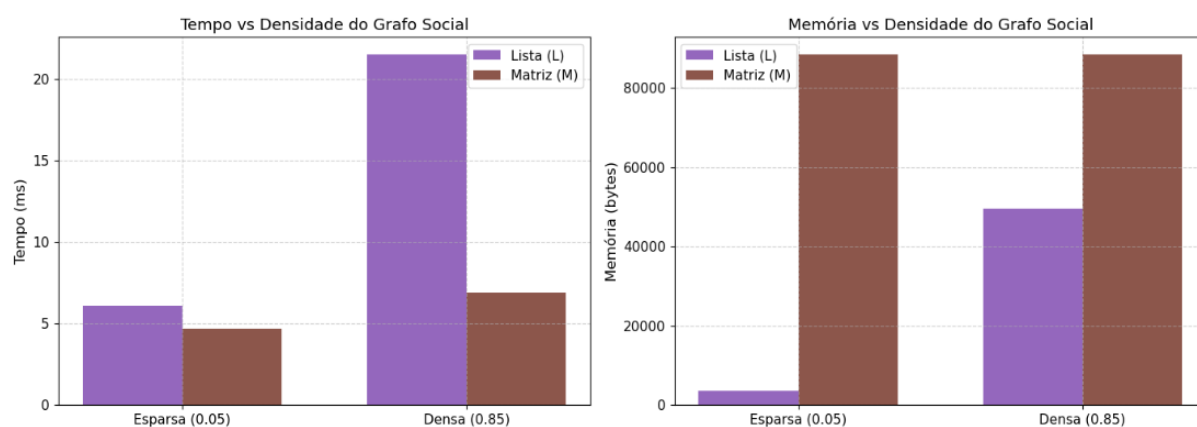
Este experimento analisou especificamente o Grafo de Temas (bipartido e não-direcionado), alterando a quantidade média de interesses vinculados a cada usuário de 2 temas para 15 temas, fixando a base em $n = 100$ e $m = 30$.



- **Eficiência de Espaço (memória):** O consumo de memória manteve-se constante em ambos os modos (9.040 bytes para L e 107.600 bytes para M). Esse fenômeno decorre do fato de que o cálculo volumétrico das estruturas base é definido pelos limites máximos de n e m cadastrados, não sofrendo flutuação direta pelo grau de conexões de interesse.
- **Eficiência de Tempo:** No modo Lista, notou-se um acréscimo no tempo de execução, subindo de 7,14 ms para 7,92 ms. Com o aumento de arestas ligando usuários a temas, as listas encadeadas baseadas no struct Nó tornam-se progressivamente maiores. Consequentemente, rotinas que dependem da varredura linear dessas cadeias enfrentam um custo computacional adicional. A Matriz manteve-se marginalmente imune a essa flutuação por trabalhar com atribuição direta.

5.3 Teste 3: Impacto das Relações de Seguimento e Densidade

A densidade do grafo social (número de arestas existentes / número máximo de arestas possíveis) é um dos fatores que mais diferenciam o desempenho prático entre listas e matrizes. Foram confrontados dois extremos com $n = 100$: uma rede altamente esparsa (densidade 0,05 com 80 arestas) e uma rede altamente densa (densidade 0,85 com 2.000 arestas).



- **Análise no cenário esparsa:** A Lista de Adjacência mostrou-se a melhor escolha estrutural. Ela registrou um tempo competitivo (6,11 ms contra 4,68 ms da Matriz)

operando com uma fração pequena de memória: somente 3.600 bytes frente aos 88.400 bytes exigidos pela Matriz.

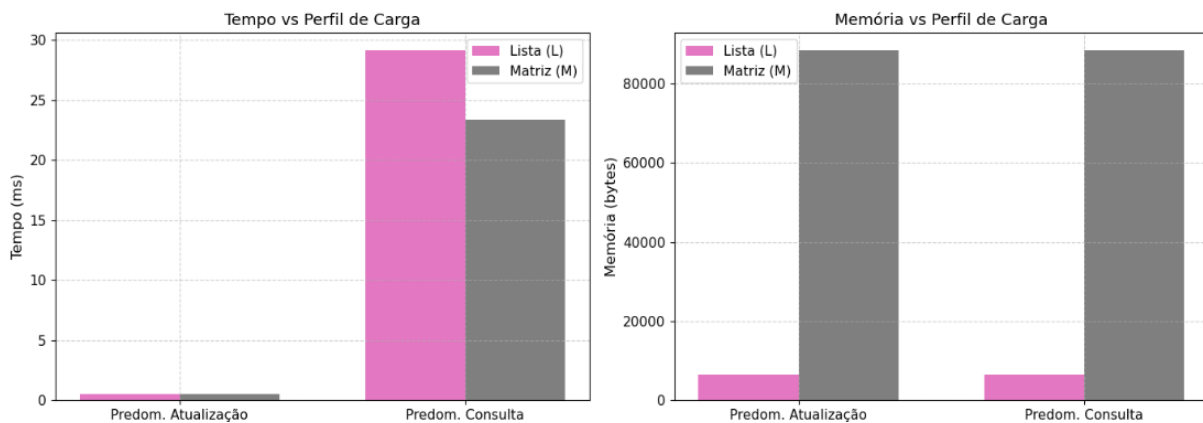
- **Análise no cenário denso:** Houve uma inversão drástica na eficiência temporal. O tempo de processamento da Lista disparou para 21,48 ms, enquanto a Matriz encerrou estavelmente em 6,89 ms.
- **Justificativa:** No grafo denso, as listas encadeadas de adjacência de cada vértice estendem-se até o tamanho limite de $O(V)$. Operações de consulta essenciais do sistema, como o método `existeAresta`, passam a exigir uma busca linear exaustiva. A Matriz de Adjacência, apesar de reter o custo fixo elevado de memória (88.400 bytes), realiza qualquer checagem ou inserção de adjacência em tempo estritamente constante $\mathcal{O}(1)$, isolando o impacto do crescimento de arestas na rede.

5.4 Teste 4: Perfis de Carga (Atualizações vs. Consultas)

O último teste avaliou o comportamento dinâmico do sistema sob o efeito dos dois perfis de carga estipulados pelo problema, fixando uma rede de $n = 100$ e 200 relações sociais.

Predominância de Atualizações: 1.000 atualizações e 10 consultas.

Predominância de Consultas: 10 atualizações e 1.000 consultas.



- **Perfil com Predominância de Atualizações:** Ambas as estruturas demonstraram alta eficiência em tempo de resposta, completando a bateria em valores inferiores a 0,60 ms. Para a Lista, isso se justifica porque a inserção de novas arestas ocorre sempre no início da lista encadeada ($\mathcal{O}(1)$). Para a Matriz, o processo resume-se a uma atribuição booleana direta na coordenada da memória ($\mathcal{O}(1)$).
- **Perfil com Predominância de Consultas:** O tempo geral de processamento elevou-se expressivamente devido à intensidade dos loops de checagem. A Lista de Adjacência demonstrou pior desempenho computacional, demandando 29,15 ms contra 23,40 ms da Matriz. A constante necessidade de iterar sequencialmente sobre os ponteiros dos nós vizinhos gera sobrecarga de processamento acumulada sob cargas massivas de leitura, cenário onde o acesso indexado por cálculo direto de memória da matriz manifesta sua superioridade assintótica.

5.5 Conclusão da Análise Experimental

A representação por Lista de Adjacência confirma-se ideal para sistemas de redes sociais reais predominantemente esparsas ou focadas em alta taxa de atualizações, destacando-se pela economia drástica de memória física.

A representação por Matriz de Adjacência consolida-se como a melhor escolha em cenários específicos de altíssima densidade (redes saturadas de conexões mútua) ou sob predominância crítica de consultas, pagando o preço de um custo fixo de memória quadrática para assegurar tempo de resposta imediato e estável.

6 Conclusão

A realização deste projeto proporcionou uma consolidação prática dos conceitos de Tipos Abstratos de Dados (TADs), gerenciamento manual de memória em C++11, ponteiros e estruturas de dados. O processo de desenvolvimento evidenciou que a escolha de uma estrutura de dados não deve ser feita de maneira arbitrária, mas sim orientada pelas restrições e requisitos do problema, como o volume de dados e o perfil de uso do sistema.

A análise experimental comprovou a relação direta entre a teoria assintótica e o comportamento computacional do mundo real. Pôde-se constatar que a Lista de Adjacência apresenta uma eficiência espacial superior ($\mathcal{O}(V + E)$), sendo a escolha ideal para redes esparsas ou sistemas com alta frequência de escritas e atualizações. Por outro lado, a Matriz de Adjacência demonstrou sua robustez em cenários de alta densidade e sob intensa carga de leitura (consultas), garantindo tempo de resposta constante ($\mathcal{O}(1)$) para a verificação de adjacências, apesar do seu custo fixo quadrático de memória ($\mathcal{O}(V^2)$).

7 Bibliografia

- ANISIO; GISELE; MÁRCIO; WAGNER. Trabalho Prático 1: Modelagem de Redes Sociais Acadêmicas. DCC221 – Estruturas de Dados. Belo Horizonte: Departamento de Ciência da Computação, Universidade Federal de Minas Gerais (UFMG), 2026.
- Slides da disciplina Estruturas de Dados - DCC/UFMG.
- CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- DROZDEK, Adam. Estrutura de Dados e Algoritmos em C++. 4. ed. Cengage Learning, 2016.
- STROUSTRUP, Bjarne. A Programação em C++: Princípios e Prática. 2. ed. Porto Alegre: Bookman, 2014.